Project 2 — The Librarian

CS496 AI Engineering

Dr Abdeldjalil Labed

Mediterranean Institute of Technology

Fall 2026



Outline

The brief

The corpus

How it works

What to measure

What you hand in

The brief

Build and **deploy** a retrieval-augmented (RAG) service over a **real collection of documents** with **real users**. Not a toy dataset.

This is the largest project. You ship a working version in **Week 7**, then spend **Week 8** making it cheaper and faster **without losing accuracy**.

- **Time:** Weeks 5–8
- **Weight:** 25% of the final grade
- **Team:** max 4 students, same team all term
- **Working version:** end of Week 7
- **Optimised version + demo:** first session of Week 9
- **Grading focus:** Works 25%, Measured 25%, Efficient 25%

Objective

A model that answers from your documents, with citations, for people who actually need the answers.

At the end of the project, every team member can:

1. **Build an index**: split documents into chunks, embed them, and store them so they can be searched.
2. **Retrieve well**: combine keyword search and vector search, then rerank the results.
3. **Answer with citations**: every claim points to the document it came from.
4. **Measure retrieval and generation separately**: recall@k for search, faithfulness for answers.
5. **Run an ablation**: show what hybrid search and the reranker each added.
6. **Optimise**: cut cost or p95 latency by 50% with at most 2 points of accuracy lost.

A good corpus: say yes to all five

1. **It is real.** Documents people use today, not a dataset from a tutorial.
2. **It has real users.** People who will ask questions and tell you if the answers are wrong.
3. **You are allowed to use it.** Public, or you have written permission. No personal data.
4. **It is big enough to need search.** Hundreds of pages or more, so the answer cannot fit in one prompt.
5. **Questions are checkable.** You can build 100+ question-answer pairs with the source passage for each.

The corpus decides the whole project. Choose it in Week 5, not Week 6.

Corpus ideas

Pick one, or propose your own. Get the instructor's OK by the end of Week 5.

- **University course catalogue and degree rules**
  real students; hard prerequisite logic
- **A lab's paper archive**
  PhD students as users; filter by year and author
- **City council minutes and bylaws**
  long documents; chunking is the whole problem
- **IT department runbooks**
  real staff; a wrong answer costs a lot
- **Tunisian legal code (Journal Officiel)**
  public, bilingual; cross-references are hard
- **An open-source project's docs, issues, discussions**
  three source types, different chunk shapes
- **Clinical guidelines for one specialty**
  faithfulness matters more than fluency
- **A newspaper's archive**
  journalists; questions about time ("in 2019…")
- **Student handbook, housing, financial aid**
  questions come from your own classmates
- **API docs for a large SDK**
  code blocks break simple chunkers; answers can be run

The pipeline

Offline: build the index

- **Load** documents and keep their metadata (title, date, author, section).
- **Chunk** them. Chunk size and overlap are decisions you must test, not defaults.
- **Embed** each chunk and store it: vector database, Postgres, or a flat file. Justify the choice.
- Also build a **keyword index** (BM25) for hybrid search.

Online: answer a question

- **Rewrite** the question if needed (short, vague, or multi-part questions).
- **Retrieve** top-k chunks with keyword + vector search, filtered by metadata.
- **Rerank** them with a cross-encoder or a small model.
- **Generate** an answer from those chunks only, with a **citation** for each claim.
- Return “I don’t know” when nothing relevant is found.

Write down the failure cases you know about. Long documents, tables, code blocks, and cross-references will all break something.

Week 8: make it cheaper and faster

The target

- **Cut cost per request or p95 latency by 50%.**
- **Lose at most 2 points** of accuracy on your own eval.
- Measure before and after with the same test set.

Where the savings usually are

- Fewer or shorter chunks sent to the model
- A smaller model for generation, or for reranking
- **Prompt caching** for the fixed part of the prompt
- Caching repeated questions
- Cheaper embeddings, or a smaller index
- Skipping the reranker when the first search is confident

Every change goes in the optimisation report with the numbers before and after.

What to measure

Retrieval

- **Recall@k**: is the right passage in the top k?
- Ablation: keyword only, vector only, hybrid, hybrid + reranker
- Latency of the search step

Generation

- **Faithfulness**: is every claim supported by a cited chunk?
- Answer correctness on your 100+ Q&A pairs (use your P1 harness)
- Citation accuracy: does the cited chunk really say that?
- "I don't know" when it should

Service

- p50 / p95 latency end to end
- Cost per request, today and at 100×
- Before / after numbers for Week 8
- Feedback from real users

Retrieval and generation are measured separately. A good answer from the wrong passage is a bug.

What you hand in

The repo

- `README.md`: one setup command, one run command, results, Contributions section
- The **deployed service** with a URL and a way to ask questions
- `data/`: eval questions with answers and source passages
- `eval/`: retrieval eval, generation eval, ablation script
- `design.md`: chunking, index choice, known failure cases
- `optimisation.md`: Week 8 report with before / after numbers
- `postmortem.md`. No API keys, no personal data

The report (max 6 pages)

1. The corpus and its users
2. Pipeline design: chunking, index, retrieval, reranking, generation
3. Retrieval results and the ablation table
4. Generation results: faithfulness and correctness
5. Week 8: what you changed and what it saved
6. Known failure cases and cost model

Postmortem: what went wrong and what you learned. A postmortem with no problems loses points.

Rules for every project: runs from a clean clone · has an eval · has a cost model · has a postmortem · you can explain every line.